

Software Design Specification

Dalitz Plot Interactive Web Visualizer

Version 1.1 - Physics formulation included

1. Vision and scope

The Dalitz Plot Interactive Web Visualizer is a web platform for constructing, visualizing, comparing, exporting and importing amplitude models for three-body decays. The application targets teaching, phenomenological exploration and rapid prototyping of amplitude-analysis models.

2. Core design principles

- Suggest, do not restrict: qrules suggests physically allowed transitions, while custom hypotheses remain possible.
- Everything physical is explicit: masses, widths, phases, magnitudes, barrier radii and lineshape choices are visible.
- Everything reactive: theoretical plots and fit fractions update automatically after model changes.
- Physics and presentation are separated: the backend owns all numerical physics; the frontend edits configuration objects.
- Reproducibility: complete models can be exported to and imported from versioned JSON files.

3. Physical conventions

3.1 Units and four-vectors

All internal calculations use GeV and GeV². The user interface displays resonance masses and widths in MeV and converts them to GeV before requests are sent to the backend.

$$p_i^\mu = (p_{\{x,i\}}, p_{\{y,i\}}, p_{\{z,i\}}, E_i)$$

The metric convention is (+, -, -, -), so the invariant mass squared of a four-vector is

$$p_i^2 = E_i^2 - |\vec{p}_i|^2 = m_i^2$$

3.2 Three-body kinematics

For a decay $P \rightarrow 1 + 2 + 3$, define the pairwise invariants

$$s_{\{12\}} = (p_1 + p_2)^2, \quad s_{\{13\}} = (p_1 + p_3)^2, \quad s_{\{23\}} = (p_2 + p_3)^2$$

They satisfy the exact identity

$$s_{\{12\}} + s_{\{13\}} + s_{\{23\}} = M^2 + m_1^2 + m_2^2 + m_3^2$$

Only two invariants are independent. The default Dalitz coordinates are (s₁₂, s₁₃), while s₂₃ is reconstructed from the identity above.

3.3 Physical Dalitz boundary

For fixed s_{12} , the allowed range in s_{13} is obtained in the rest frame of the (12) pair. The relevant energies and momenta are

$$E_{1*} = (s_{12} + m_1^2 - m_2^2)/(2\sqrt{s_{12}})$$

$$E_{3*} = (M^2 - s_{12} - m_3^2)/(2\sqrt{s_{12}})$$

$$q = \sqrt{\lambda(s_{12}, m_1^2, m_2^2)}/(2\sqrt{s_{12}}), \quad p = \sqrt{\lambda(M^2, s_{12}, m_3^2)}/(2\sqrt{s_{12}})$$

where the Källén function is

$$\lambda(x, y, z) = x^2 + y^2 + z^2 - 2xy - 2xz - 2yz$$

The physical limits are

$$s_{13}^{\pm}(s_{12}) = m_1^2 + m_3^2 + 2(E_{1*}E_{3*} \pm qp)$$

4. Isobar amplitude model

4.1 Total amplitude

The total complex amplitude is written as a coherent sum

$$A(\Phi_3) = \sum_r c_r \hat{F}_r(\Phi_3) + c_{NR} \hat{F}_{NR}(\Phi_3)$$

with complex coefficients in polar form

$$c_r = a_r \exp(i\varphi_r)$$

The user edits the magnitude a_r and phase φ_r . The spin is fixed by the particle definition provided by particle/qrules and is not editable.

4.2 Resonant contribution

A resonant isobar component is factorized as

$$F_r = F_B(p) F_R(q) Z_L(\vec{p}, \vec{q}) R_r(s_{ij})$$

Here R_r is the lineshape, F_B and F_R are Blatt-Weisskopf barrier factors at the parent and resonance vertices, and Z_L is the angular term.

4.3 Relativistic Breit-Wigner

The implemented dynamic relativistic Breit-Wigner is

$$R_{RBW}(s) = 1 / [m_0^2 - s - i m_0 \Gamma(s)]$$

The mass-dependent width is

$$\Gamma(s) = \Gamma_0 (q/q_0)^{2L+1} (m_0/\sqrt{s}) [F_R(q)/F_R(q_0)]^2$$

q is the breakup momentum of the resonance daughters at invariant mass $\sqrt{s}$, and q_0 is q evaluated at $s = m_0^2$. Below threshold, the implementation must follow an explicitly documented continuation or return zero according to the selected numerical policy.

4.4 Blatt-Weisskopf barrier factors

The barrier factors are functions of $z = (rq)^2$ and $z_0 = (rq_0)^2$. The current implementation supports $L = 0, 1$ and 2 .

$$F_0(q) = 1$$

$$F_1(q) = \sqrt{[(1 + z_0)/(1 + z)]}$$

$$F_2(q) = \sqrt{[(z_0^2 + 3z_0 + 9)/(z^2 + 3z + 9)]}$$

Separate radius parameters are used for the resonance and parent vertices. The default values are $r_R = 1.5 \text{ GeV}^{-1}$ and $r_B = 5.0 \text{ GeV}^{-1}$, both editable.

4.5 Angular distribution

The angular term is evaluated using the Zemach-tensor convention in the resonance rest frame. For orbital angular momentum $L = 0, 1$ and 2 :

$$Z_0 = 1$$

$$Z_1 = -2 \vec{p} \cdot \vec{q}$$

$$Z_2 = (4/3)[3(\vec{p} \cdot \vec{q})^2 - |\vec{p}|^2 |\vec{q}|^2]$$

The ordering of the two particles inside the selected pair defines the sign convention for odd spin. This convention must remain fixed across plotting, toy generation, import/export and testing.

4.6 Non-resonant term

The initial non-resonant model is a constant scalar amplitude

$$F_{NR}(\Phi_3) = 1$$

It is normalized with the same phase-space convention as the resonant components and interferes coherently with them. Future extensions may include exponential or polynomial non-resonant models.

5. Identical-particle symmetrization

If the final state contains identical particles, symmetrization is applied to the complex amplitude before taking the modulus squared.

$$F_r^{\wedge \text{sym}}(\Phi_3) = \sum_{\pi \in P_{\text{id}}} F_r(\pi \cdot \Phi_3)$$

$$I(\Phi_3) = |A_{\text{sym}}(\Phi_3)|^2$$

Only distinct isobar assignments are retained. Permutations that exchange the two daughters inside the same isobar without creating a distinct physical assignment are removed to avoid double counting.

6. Component normalization

Each complete component, including lineshape, barrier factors, angular term and all required symmetrized assignments, is normalized over the physical three-body phase space.

$$N_r = \int d\Phi_3 |F_r^{\wedge \text{sym}}(\Phi_3)|^2$$

$$\hat{F}_r(\Phi_3) = F_r^{\text{sym}}(\Phi_3) / \sqrt{N_r}$$

$$\int d\Phi_3 |\hat{F}_r(\Phi_3)|^2 = 1$$

The normalization integral is evaluated numerically with a phase-space sample. It is cached and recalculated only when a physical shape parameter changes. Changing only magnitude or phase does not require a new normalization.

7. Intensity and fit fractions

7.1 Intensity

$$I(\Phi_3) = |A(\Phi_3)|^2$$

The theoretical Dalitz plot evaluates this function on a deterministic grid inside the physical boundary. The three theoretical projections are obtained by numerical integration over the complementary coordinate.

7.2 Fit fractions

The fit fraction of component r is

$$FF_r = [\int d\Phi_3 |c_r \hat{F}_r(\Phi_3)|^2] / [\int d\Phi_3 |A(\Phi_3)|^2]$$

Interference terms appear only in the denominator. Consequently, the sum of fit fractions is not required to equal 100%; it may be larger or smaller depending on constructive or destructive interference.

8. Phase-space generation and toy Monte Carlo

The phasespace library generates four-momenta distributed over Lorentz-invariant phase space. For each generated point, the application evaluates the dynamic weight

$$w_{\text{dyn}} = |A(\Phi_3)|^2$$

The optional toy section is separate from the automatically updated theoretical plots. When requested, it displays a Dalitz distribution and three invariant-mass histograms based on the generated sample.

CSV export contains only:

- p1_px, p1_py, p1_pz, p1_E
- p2_px, p2_py, p2_pz, p2_E
- p3_px, p3_py, p3_pz, p3_E
- s12, s13, s23
- dynamic_weight = $|A|^2$

9. Particle data and reaction validation

The particle library provides particle masses, widths, PDG identifiers, spins and other quantum numbers. Masses and widths are displayed in MeV and converted internally to GeV.

qrules validates the selected mother and three-particle final state and generates suggested intermediate states. A channel is revalidated whenever a model is imported.

Custom states remain possible. The application warns when an amplitude is outside the qrules suggestion set, but does not prohibit exploratory hypotheses.

10. Model persistence

A complete model can be exported to and imported from JSON. The format includes a schema version, decay channel, symmetrization flag, selected amplitudes, lineshape parameters, magnitudes, phases, barrier radii, plot resolution, toy settings and seed.

During import, the backend validates the schema and channel. Spin values in the file are treated as reference only; the authoritative spin is obtained from particle/qrules.

11. Numerical and validation requirements

- Verify four-momentum conservation event by event in the parent rest frame.
- Verify $s_{12} + s_{13} + s_{23} = M^2 + m_1^2 + m_2^2 + m_3^2$ within floating-point tolerance.
- Test invariance under exchange of identical particles.
- Test that every normalized component integrates to unity within Monte Carlo uncertainty.
- Test phase conventions for odd-spin amplitudes.
- Test fit fractions against direct numerical integration.
- Use deterministic seeds for regression tests.
- Keep unit conversions at API boundaries only.

12. Planned physics extensions

- Gounaris-Sakurai lineshape for vector resonances.
- Flatté lineshape with channel-dependent couplings and thresholds.
- LASS parametrization for $K\pi$ S-wave.
- K-matrix and coupled-channel models.
- Alternative angular conventions with explicit convention metadata.
- Efficiency maps and background models.
- CP-conjugate models and local asymmetry visualization.

13. Software architecture summary

- Frontend: React, TypeScript and Plotly.
- Backend: FastAPI and Pydantic.
- Particle data and quantum numbers: particle.
- Reaction validation and resonance suggestions: qrules.
- Phase-space generation: phasespace.
- Numerical evaluation: NumPy and SciPy.
- Model exchange: versioned JSON.
- Toy export: CSV.